BCA309T: Cryptography & Network Security

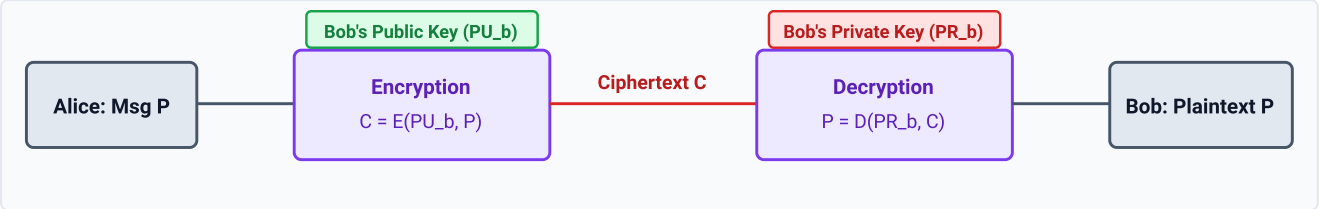
ACADEMIC SOLUTIONS MANUAL

Assignment 5: Introduction to Asymmetric Cryptography (Complete Solutions)

Q1 [10 Marks] Asymmetric Principles, Dual-Key Roles, and Security Services

Asymmetric (public-key) cryptography uses mathematically linked, non-identical key pairs: a **Public Key** (freely published) and a **Private Key** (kept secret by the owner). It achieves three fundamental services:

- **Confidentiality:** Sender encrypts payload with *Recipient's Public Key*. Only the recipient's matching *Private Key* can invert the function to decrypt.
- **Authentication & Non-Repudiation:** Sender encrypts a hash with their own *Private Key* (Digital Signature). Anyone can verify using the sender's *Public Key*, proving identity and preventing denial of origin.



Q2 [8 Marks] Symmetric vs. Asymmetric Cryptography: Comparative Table

Heading	Symmetric Cryptography	Asymmetric Cryptography
Number of Keys	One shared secret key for encryption and decryption.	Two mathematically related keys (Public Key and Private Key).
Key Distribution	Challenging; requires physically secure or pre-shared channels.	Straightforward; public keys are published openly without secrecy.
Operational Speed	Very fast; suitable for bulk streaming and gigabit links.	Slow (~1000× slower); computation heavy modular exponentiation.
Key Length (Equivalent)	128 bits or 256 bits (AES) for standard security.	2048 to 4096 bits (RSA) for equivalent security level.
Scalability for n Users	Poor: requires $n(n-1)/2$ keys (quadratic growth).	Excellent: requires $2n$ keys total (n public, n private).
Typical Applications	Bulk payload encryption (AES-GCM), disk encryption (BitLocker).	Key exchange (Diffie-Hellman), digital certificates, signatures (RSA).

Q3 [10 Marks] Core Building Blocks of Public-Key Cryptography

- **Trapdoor One-Way Functions:** Functions Y = f(X) that are easy to compute in one direction, but computationally infeasible to invert ($X = f^{-1}(Y)$) unless a specific secret piece of auxiliary information (the "trapdoor") is known. Used to establish cryptographic asymmetry.
- **Diffie-Hellman Key Exchange (1976):** Allows two parties sharing no prior secrets to establish a shared symmetric session key over an insecure channel by exchanging $g^a \bmod p$ and $g^b \bmod p$; grounded in the computational intractability of the *Discrete Logarithm Problem*.
- **Digital Signatures:** Provides data integrity, entity authentication, and non-repudiation. The sender creates a cryptographic digest of the message and encrypts it with their private key: S = D(PR, H(M)). Verification is carried out using the sender's public key.
- **Public Key Infrastructure (PKI) & Digital Certificates:** Solves the man-in-the-middle vulnerability by having a trusted Certificate Authority (CA) digitally sign an X.509 certificate binding an entity's identity to their public key.

Q4 [10 Marks] Historical Development of Public-Key Cryptography

- **Key Distribution Problem:** Symmetric systems faced a catastrophic scaling limit: establishing secure pairwise keys across millions of global hosts required an impossible pre-shared infrastructure.
- **Diffie and Hellman (1976):** Published *"New Directions in Cryptography,"* introducing the theoretical framework of public-key cryptography and the DH key exchange protocol.
- **RSA Invention (1977):** Ron Rivest, Adi Shamir, and Leonard Adleman at MIT instantiated the first practical general-purpose asymmetric algorithm supporting both encryption and digital signatures based on prime factorization.
- **Classified GCHQ Predecessors (1969–1973):** Declassified in 1997, British intelligence researchers James Ellis (concept of "non-secret encryption"), Clifford Cocks (equivalent of RSA), and Malcolm Williamson (equivalent of DH) discovered public-key cryptography years earlier.
- **Later Schemes:** Taher ElGamal (1985) introduced the ElGamal system based on discrete logs. Neal Koblitz and Victor Miller (1985) formulated *Elliptic Curve Cryptography (ECC)*, delivering equivalent security with drastically smaller key sizes.

Q5 [8 Marks] Hybrid Cryptosystems: Real-World Architecture (TLS Example)

Asymmetric algorithms are mathematically rich but computationally intensive (excessive CPU load and latency for large payloads). Symmetric ciphers are fast but suffer from key exchange vulnerabilities. **Real-world systems deploy Hybrid Cryptography:**

- **TLS Handshake Example:** The client and server use public-key cryptography (RSA or ECDHE) to authenticate certificates and negotiate a random, short-lived 256-bit symmetric "session key."
- **Data Transfer Phase:** Once negotiated, high-speed symmetric ciphers (AES-GCM or ChaCha20-Poly1305) encrypt all application data. This combines the robust key-distribution of asymmetric cryptography with the raw throughput of symmetric encryption.

Q6 [12 MARKS] THE RSA ALGORITHM: KEY GENERATION, MATH & FLOWCHART

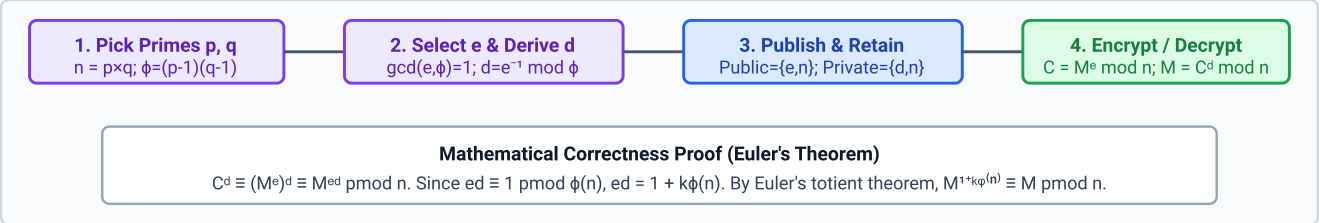
The RSA cryptosystem operates on modular arithmetic over the ring of integers modulo n.

Step-by-Step Key Generation:

1. Select two distinct, large prime numbers p and q (typically 1024 or 2048 bits each).
2. Calculate the modulus: $n = p \times q$.
3. Calculate Euler's totient function: $\phi(n) = (p - 1)(q - 1)$.
4. Select integer e (public exponent) such that $1 < e < \phi(n)$ and $\gcd(e, \phi(n)) = 1$ (commonly e = 65537).
5. Calculate private exponent d using the Extended Euclidean Algorithm: $d \equiv e^{-1} \pmod{\phi(n)}$, meaning $d \times e \equiv 1 \pmod{\phi(n)}$.
6. **Public Key:** PU = {e, n}. **Private Key:** PR = {d, n}. (Discard p, q, $\phi(n)$ securely).

Encryption Formula: Plaintext $M < n$ is encrypted as: $C = M^e \bmod n$.

Decryption Formula: Ciphertext C is decrypted as: $M = C^d \bmod n$.



Proof of Decryption Correctness: Since $d \times e \equiv 1 \pmod{\phi(n)}$, there exists integer k such that $ed = k \cdot \phi(n) + 1$. Substituting $C = M^e \bmod n$ into decryption gives: $C^d \equiv (M^e)^d \equiv M^{ed} \equiv M^{k \cdot \phi(n) + 1} \equiv (M^{\phi(n)})^k \cdot M \pmod n$. By Euler's Theorem, for any M coprime to n, $M^{\phi(n)} \equiv 1 \pmod n$. Thus: $C^d \equiv (1)^k \cdot M \equiv M \pmod n$. (The identity also holds when $\gcd(M, n) \ne 1$ via the Chinese Remainder Theorem).

Q7 [10 Marks] RSA Key Sizes, Factoring Hardness, ECC Comparison & Quantum Threats

Key Size Definition: The bit length of the modulus n (e.g., a 2048-bit RSA key has an n of 2048 binary bits). Security relies on the asymmetry between multiplication (trivial: $O(b^2)$) and prime factorization using the General Number Field Sieve (GNFS, sub-exponential complexity).

- **Recommended Bit Lengths: 1024-bit:** Deprecated/insecure since 2010; broken by nation-states. **2048-bit:** Current minimum standard for legacy compatibility through 2030. **3072-bit & 4096-bit:** Highly secure for root CAs and sensitive government infrastructure.
- **Comparison with ECC:** ECC achieves equivalent security with vastly smaller keys due to the lack of sub-exponential algorithms for elliptic curve discrete logarithms: *RSA-2048 ≈ ECC-256 bits; RSA-3072 ≈ ECC-384 bits; RSA-15360 ≈ ECC-512 bits*. ECC uses significantly less bandwidth and battery.
- **Quantum Threat (Shor's Algorithm):** On a sufficiently large fault-tolerant quantum computer, Peter Shor's polynomial-time algorithm solves prime factorization and discrete logs in $O(b^3)$ time, completely breaking RSA and ECC. The cryptographic industry is transitioning to Post-Quantum Cryptography (NIST standards: ML-KEM, ML-DSA).

Q8 [12 Marks] Security Attacks on RSA, OAEP Padding & Best Deployment Practices

- **Factoring Attacks:** Using GNFS to split n into p and q. Mitigated by using n \ge 2048 bits and cryptographically random prime selection.
- **Small Public Exponent Attack:** If e = 3 and the same message is sent to 3 parties, Hastad's Broadcast Attack uses the Chinese Remainder Theorem to recover M via integer cube roots.
- **Common Modulus Attack:** If an organization assigns different e_1, e_2 to users under the same n, an adversary can recover plaintext without private keys using Bézout's identity.
- **Timing & Side-Channel Attacks:** Kocher's attack measures execution time variations in modular exponentiation to extract private key bits. Mitigated by *cryptographic blinding* ($C' = C \cdot r^e \bmod n$).
- **Chosen-Ciphertext Attack (Bleichenbacher's Attack):** Exploits RSA padding error messages to decrypt SSL sessions.
- **OAEP Padding:** Textbook RSA is deterministic ($M^e \bmod n$ always yields identical C) and homomorphic. *Optimal Asymmetric Encryption Padding (OAEP)* adds randomized two-round Feistel padding to achieve IND-CCA2 (ciphertext indistinguishability under adaptive chosen-ciphertext attack).

Deployment Best Practices: (1) Never use textbook RSA; enforce RSA-OAEP for encryption and RSA-PSS for signatures; (2) Use minimum 2048-bit modulus; (3) Set public exponent e = 65537 ($2^{16}+1$); (4) Implement constant-time code with blinding; (5) Distribute public keys via certified X.509 PKI.